# 作业一

一、假设某科学计算程序在单核上的执行时间为 $T_{serial} = 2000$秒。该程序中 90% 的计算任务可以完全并行化。现将其部署在拥有 p 个处理器的分布式集群上。经实测，该算法在并行化后会产生额外的通信延迟，延迟时间与处理器数量的关系公式为$T_{comm} = 0.05 \times p^2$秒（此开销属于非并行部分）

1. 若不计通信延迟，根据阿姆达尔定律计算该程序在 p=4, 8, 16 时的理论加速比

2. 计入通信延迟$T_{comm}$后，写出并行运行时间$T_{parallel}$随 p 变化的数学表达式

3. 计算在计入通信延迟的情况下，处理器数量 p 取何值时能获得最大加速比？此时的加速比是多少？

答案：

1. 根据阿姆达尔定律公式$S = \frac{1}{(1-f)+f/p}$，其中$f = 0.90, (1-f) = 0.10$

p=4 时：$S = \frac{1}{0.10+0.90/4} = \frac{1}{0.325} \approx 3.08$

p=8 时：$S = \frac{1}{0.10+0.90/8} = \frac{1}{0.2125} \approx 4.71$

p=16 时：$S = \frac{1}{0.10+0.90/16} = \frac{1}{0.15625} \approx 6.40$

2. $T_{parallel}(p) = T_{serial} \times (1-f) + \frac{T_{serial} \times f}{p} + T_{comm}$

代入通信延迟与处理器数量的关系公式以及串行执行时间得

$$T_{parallel}(p) = 200 + \frac{1800}{p} + 0.05p^2$$

3. 计算最大加速比，即要使$T_{parallel}(p)$最小，将上一问中得到的$T_{parallel}(p)$公式对 p 求一阶导$\frac{dT_{parallel}}{dp} = -\frac{1800}{p^2} + 0.1p = 0$

得$0.1p = \frac{1800}{p^2}$，$p = \sqrt[3]{18000} \approx 26.21$，取 p 为 26

$T_{parallel}(26) = 200 + \frac{1800}{26} + 0.05 \times 26^2 = 200 + 69.23 + 33.8 = 303.03$秒

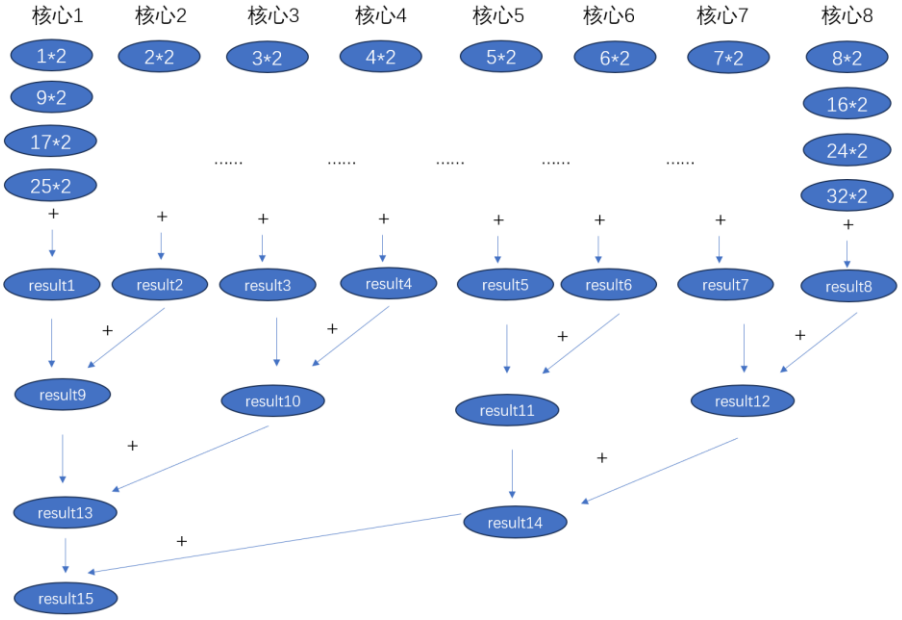
最大加速比$S_{max} = \frac{2000}{303.03} \approx 6.60$

二、下面是一段简单的代码，试分析在单核模式下和多核模式下的性能对比：

假设核心数为 8，并行计算时负载均衡，采用树形规约，每次计算耗时和规约耗时均相同，时间均为$t$，分析完成整个计算时，单核与多核情况下主核分别进行了多少次运算，两者完成整个计算耗时对比，并简单画出多核计算时的树形图或者示意图。

```
int total_sum=0;
    for(int i=1;i<=32;i++){
        total_sum+=i*2;
    }
```

答案：在进行单核计算时，需要对每个 i 进行一次乘法运算，总共需要 32 次乘法运算，然后再将每个数相加，再需要 32 次加法运算，一共 64t 的计算耗时。在 8 核负载均衡的并行计算模式下，每个核心首先分配 4 个乘法和 4 个加法，然后每两个核心将自己的计算结果进行规约相加，在这步中主核心只需要进行三次规约操作即可，主核一共只需要进行 11t 的时间，单核与多核的计算时间之比为 64:11。




三、考虑一个现代化的大规模异构超级计算机节点，其硬件配置如下：

主机端：包含 2 颗多核 CPU（每颗 64 核），支持 AVX-512 向量指令集。

加速端：挂载 4 张高性能 GPU 卡。

互联：CPU 与 GPU 之间通过 PCIe 5.0 连接，GPU 与 GPU 之间通过 NVLink 互联。

该节点正在运行一个大规模流体动力学模拟程序，请回答以下问题：

**（1）**请分别指出以下三个层级在 Flynn 分类法中所属的类别：

(a) 单个 CPU 核心执行 AVX-512 向量指令时。

(b) 单张 GPU 内部的数千个 CUDA 核心协同工作时。

(c) 整个节点（2 颗 CPU + 4 张 GPU）协同处理不同任务模块时。

**（2）**在该流体模拟中，为了提高容错性，工程师提议让 3 个不同的 CPU 核心同时读取相同的输入数据流，分别运行三种不同的误差校正算法，最后通过投票机制输出结果，这一特定子任务属于 Flynn 分类法中的哪一类？学术界常争论 MISD 在现实中几乎不存在，请结合此案例反驳或支持该观点，并说明 MISD 架构在现代计算中的主要应用场景是什么？

**（3）**传统的 Flynn 分类法将系统严格划分为四类，然而，现代超算节点通常被称为 MIMD-SIMD 混合架构，为什么不能简单地将整个节点归类为单纯的 MIMD 或 SIMD?

答案：

**（1）**

（a）SIMD，控制单元发出一条向量指令，该指令被解码后，同时作用于寄存器中的多个数据元素，指令流是单一的，数据流是多路的。

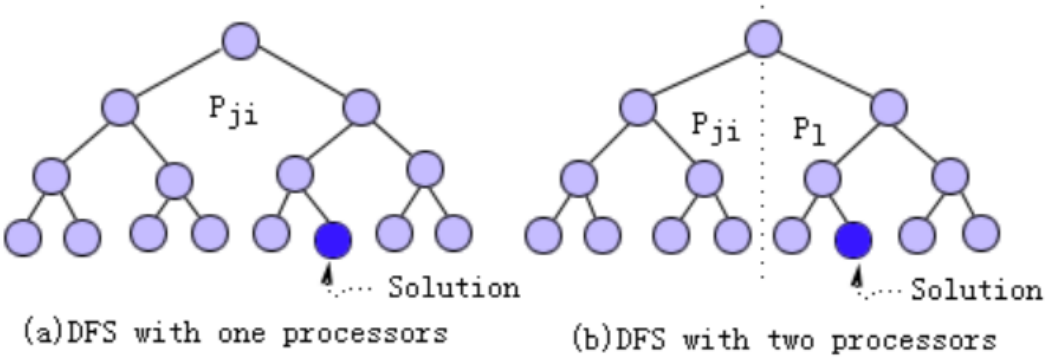
（b）SIMD（或者写 SIMT），GPU 内部的并行计算架构使用的是 SIMD 架构，一条指令可以由多个核心一起进行计算。

（c）MIMD，CPU 的不同核心可以执行完全不同的代码，CPU 和 GPU 也可以执行不同的程序段，它们拥有独立的指令流和独立的数据流，异步执行。

**（2）**该子任务属于 MISD，输入数据流是相同的，但三个核心执行的是不同的指令流。MISD 并非不存在，只是不用于常规的性能加速，因为跑不同算法通常比跑同一算法慢，其主要用于容错和安全关键系统场景，在这些场景下，正确性大于速度。

（3）节点间的调度、CPU 核心的独立任务分发体现了 MIMD 的灵活性，可以高效并行处理任务，而节点内的密集计算利用了 SIMD 的高能效比，能够大大提高数据计算的速度，单纯叫 MIMD 忽略了向量并行计算的巨大贡献，单纯叫 SIMD 忽略了复杂的控制流和任务调度。

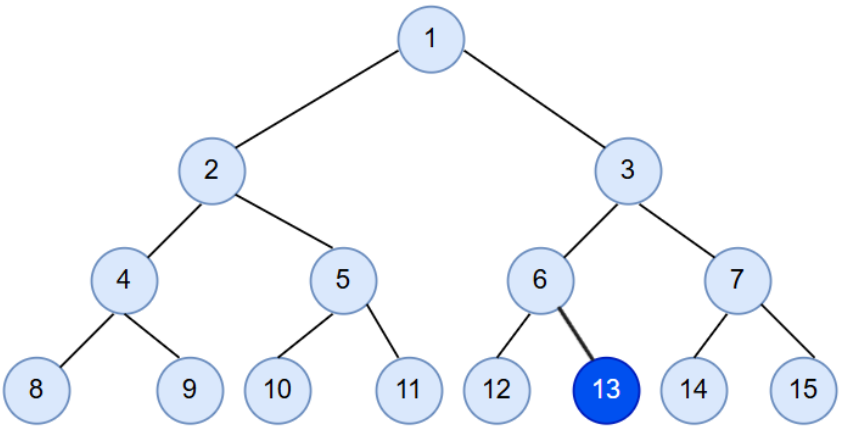
四、考虑下图所示的搜索树，其中深色的节点表示问题的解




1. 对树的串行搜索采用深度优先（DFS）算法，如果遍历树的每个边耗费一个单位时间，那么需要多长时间才能找到解？
2. 将树在两个处理器间进行分布，如图所示，共同完成搜索任务，如果两个处理器都在它们各自的树上进行 DFS 搜索（均从根节点开始）。假设两个处理器速度相当均为每个边耗费一个单位时间，那么要找到解需要多长时间，加速比是多少？

答案：

将树的节点如下进行编号




1. 串行搜索算法的节点访问顺序为：

1→2→4→8→4→9→4→2→5→10→5→11→5→2→1→3→6→12→6→13

共需要访问 19 个边，搜索时间为 19 个单位时间

2.并行搜索时，

第一个处理器的搜索顺序为：

1→2→4→8→4→9→4→2→5→10→5→11

第二个处理器的搜索顺序为：

1→3→6→12→6→13

5 个单位时间后，第二个处理器已经找到解，算法结束

并行算法的加速比：

$$S = \frac{19}{5} = 3.8$$

五、设有二维数组 A[8][8]，按行优先方式存放在内存中。假设 cache 一次可存放一整行数据。比较下面两段程序的访存行为。

程序一：

```
for (int i = 0; i < 8; i++) {
    for (int j = 0; j < 8; j++) {
        sum += A[i][j];
    }
}
```

程序二：

```
for (int j = 0; j < 8; j++) {
    for (int i = 0; i < 8; i++) {
        sum += A[i][j];
    }
}
```

请回答下列问题：

（1）哪一段程序的缓存访问效率更高？为什么？

（2）两段程序分别体现了怎样的空间局部性特征？

（3）若数组规模进一步增大，而 cache 大小不变，两段程序的性能差异会如何变化？请简要说明理由。

参考答案：

（1）程序一的缓存访问效率更高。因为二维数组按行优先存储，同一行元素在内存中连续存放。程序一按行访问，访问顺序与存储顺序一致，更容易充分利用 cache；程序二按列访问，相邻访问元素在内存中距离较远，容易发生更多 cache miss。

（2）程序一具有较强的空间局部性，因为连续访问同一行中相邻元素。

程序二的空间局部性较差，因为访问顺序是按列进行，相邻两次访问的数据在内存中并不相邻。

（3）若数组规模继续增大，而 cache 大小保持不变，则两段程序的性能差异通常会进一步增大。因为程序一仍能较好利用顺序访问带来的局部性，而程序二会因跨行访问导致更多 cache miss，访存代价更高。

六、某同学想测试程序 test_prog 的运行时间，分别使用了两组输入数据 input1 和 input2，并使用 Unix shell 命令 time 得到如下结果：

对于 input1：

```
$ time ./test_prog < input1

real 0m0.002s
user 0m0.001s
sys  0m0.000s
```

对于 input2：

```
$ time ./test_prog < input2

real 0m1.532s
user 0m1.210s
sys  0m0.120s
```

其中，real 为程序从开始到结束的总耗时

user 为程序在用户态执行所消耗的 CPU 时间

sys 为程序在内核态执行所消耗的 CPU 时间，例如 I/O、系统调用等带来的时间

请回答下列问题：

（1）对于哪一组输入数据，更适合直接使用这种方式测量程序运行时间？为什么？
（2）如果程序运行时间非常短，计时可能会遇到什么问题？应如何改进测量方法？

参考答案：

（1）更适合直接使用这种方式测量的是 input2。因为其运行时间较长，计时结果更稳定，计时器分辨率带来的相对误差较小。对于 input1，程序运行时间太短，单次测试结果容易受到系统调度、计时精度等因素影响。

（2）如果程序运行时间非常短，则可能出现计时分辨率不足的问题，使得误差占比很大，导致结果不可靠。常见改进方法有：将程序重复执行多次，再取平均值；增大测试规模；使用更高精度的计时函数进行测量。

# 作业二

一、 死锁是并行程序中的典型错误之一，以下代码段实现了在两个进程上交换各自长度为 N 的浮点数组的功能

```
float send_buf[N], recv_buf[N];

if (my_rank == 0) {
    // 进程 0
    MPI_Send(send_buf, N, MPI_FLOAT, 1, 0, MPI_COMM_WORLD);
    MPI_Recv(recv_buf, N, MPI_FLOAT, 1, 1, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);
} else if (my_rank == 1) {
    // 进程 1
    MPI_Send(send_buf, N, MPI_FLOAT, 0, 1, MPI_COMM_WORLD);
    MPI_Recv(recv_buf, N, MPI_FLOAT, 0, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);
}
```

1. 分析该程序为什么在 N 较小时能够成功运行，而当 N 的取值非常大时，有可能永久挂起
2. 请给出一种可行的解决方案并简要说明

**答案：**

1. MPI 标准中对于较小的消息，MPI 系统通常会将其直接复制到系统预留的内部缓冲区中然后发送函数立即返回，但是当消息大小超过某一阈值时，MPI_Send 必须阻塞并等待接收端发起对应的 MPI_Recv 后才能继续。该代码段中如果消息很大两个进程都会陷入阻塞等待对方发起 MPI_Recv。然而，由于接收逻辑排在发送逻辑之后，两个进程都无法到达接收阶段从而形成死锁
2. a.可以使用 MPI_Sendrecv 替代独立的发送和接收
   b.调整两个进程的收发顺序，交错收发即可

二、 在分布式算法中，有时我们需要实现类似于 MPI_Allgather 的功能，即每个进程都有一个局部值，最后每个进程都要获得所有进程的值，实现全局数据同步，使用 C 语言编写一段符合以下要求的程序：

1. 在一个包含 size 个进程的环形传递结构（即进程 q 总是将消息发送给 (q+1) mod P，并从 (q-1+P) mod P 接收消息，其中 P 为进程总数）
）中，实现 2 中的要求
2. 每个进程持有一个整数 my_val。经过若干步迭代，每个进程计算出环

内所有值的累加和

3. 使用 MPI_Sendrecv 以避免死锁

**答案：**

```
#include <mpi.h>
#include <stdio.h>

int main(void) {
    int rank, size;
    int my_val, ring_sum, send_tmp, recv_tmp;
    int left_neighbor, right_neighbor;

    MPI_Init(NULL, NULL);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    // 计算发送与接收的目标
    right_neighbor = (rank + 1) % size;
    left_neighbor = (rank - 1 + size) % size;

    ring_sum = my_val;
    send_tmp = my_val;

    // 循环 size-1 次使数据遍历全环
    for (int i = 0; i < size - 1; i++) {
        // 使用 MPI_Sendrecv 实现同步收发，规避死锁
        MPI_Sendrecv(&send_tmp, 1, MPI_INT, right_neighbor, 0,
                     &recv_tmp, 1, MPI_INT, left_neighbor, 0,
                     MPI_COMM_WORLD, MPI_STATUS_IGNORE);

        ring_sum += recv_tmp;
        send_tmp = recv_tmp; // 将刚收到的数据传给接收对象
    } //循环结束，ring_sum 中保存所有值的累加和

    MPI_Finalize();
    return 0;
}
```

三、有两个长度为 (n=12) 的整数向量 x 和 y，其初值如下：

x = [1, 3, 5, 7, 9, 11, 13, 15, 17, 19, 21, 23]

y = [2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24]

现使用 comm_sz = 4 个 MPI 进程完成向量加法，要求如下：

（1）由 0 号进程初始化向量 x 和 y；
（2）使用 MPI 通信操作，将两个向量平均分发给各个进程；
（3）每个进程计算自己负责部分的局部和；
（4）再将结果收集到 0 号进程；
（5）最后由 0 号进程输出结果向量 z。

请回答下列问题：

（1）每个进程分别处理向量中的哪几个元素？
（2）最终结果向量 z 是什么？
（3）写出完成该任务时需要用到的主要 MPI 函数名称，并简要说明它们的作用。

**答案：**

（1）由于 (n=12)，comm_sz = 4，因此每个进程处理 (12/4=3) 个元素。

0 号进程处理：x[0..2] 和 y[0..2]

1 号进程处理：x[3..5] 和 y[3..5]

2 号进程处理：x[6..8] 和 y[6..8]

3 号进程处理：x[9..11] 和 y[9..11]

（2）逐元素相加得到：
z = x + y = [3, 7, 11, 15, 19, 23, 27, 31, 35, 39, 43, 47]

（3）主要 MPI 函数包括：

MPI_Init：初始化 MPI 环境

MPI_Comm_rank：获取当前进程编号

MPI_Comm_size：获取总进程数

MPI_Scatter：将 0 号进程中的向量数据分发给各进程

MPI_Gather：将各进程计算得到的局部结果收集到 0 号进程

MPI_Finalize：结束 MPI 环境

四、设有一个长度为 (n=10) 的整数数组 a：

a = [5, 8, 2, 6, 1, 9, 4, 7, 3, 10]

现在希望使用 comm_sz = 4 个 MPI 进程对该数组中的元素分别乘以 2，最后再把结果收集回 0 号进程。

已知 MPI_Scatter 和 MPI_Gather 的一个限制是：**每个进程接收或发送的数据个数必须相同。**

请回答下列问题：

（1）为什么在本题中不能直接使用一次 MPI_Scatter 和一次 MPI_Gather 完成任务？
（2）如果采用“前 2 个进程各处理 3 个元素，后 2 个进程各处理 2 个元素”的分配方案，请写出各进程负责的数据范围。
（3）给出一种可行的修改思路，使程序在 (n) 不能被 comm_sz 整除时仍能正确完成计算。

**答案：**

（1）因为这里 (n=10)，comm_sz = 4，而 (10/4) 不是整数。
若直接使用 MPI_Scatter 和 MPI_Gather，则要求每个进程接收和发送的数据个数相同，但本题中无法把 10 个元素平均分成 4 份，因此不能直接只靠一次标准的 MPI_Scatter 和 MPI_Gather 完成。

（2）按照“前 2 个进程各处理 3 个元素，后 2 个进程各处理 2 个元素”的方案：

0 号进程处理：a[0..2] = [5, 8, 2]

1 号进程处理：a[3..5] = [6, 1, 9]

2 号进程处理：a[6..7] = [4, 7]

3 号进程处理：a[8..9] = [3, 10]

（3）一种可行思路是：

由 0 号进程先计算每个进程应分得的数据个数和起始下标；

再使用**多次点对点通信**（如 MPI_Send / MPI_Recv）分别发送不同数量的数据；

各进程完成局部计算后，再用点对点通信把结果发回 0 号进程；或者使用支持“不同进程接收不同数量数据”的更灵活通信方式。

这样即使（n）不能被 comm_sz 整除，也能保证所有元素都被处理且不会遗漏。

五、考虑使用 MPI 实现数值积分的梯形法则，假设函数$f(x)$在区间$[a，b]$上是连续的，我们将区间划分成$n$个等宽的小梯形，每个梯形的宽度为$h=\frac{b-a}{n}$，积分的近似公式为：$\int_a^b f(x)dx \approx h[\frac{f(a)}{2}+\sum_{i=1}^{n-1} f(a+i*h)+\frac{f(a)}{2}]$,现在你编写了一个 MPI 程序，使用$p$个进程来并行计算该积分，每个进程负责计算一部分梯形的面积，然后通过 MPI_Reduce 将局部结果汇总到进程 0，最后由进程 0 输出总积分值。

（1） 假设$n$能被$p$整除，请写出每个进程负责的梯形数量、起始点和结束点的计算公式，如果$n$不能被$p$整除，每个进程负责的梯形数量又是多少。

（2） 使用 MPI_Reduce 汇总结果，该集合通信的时间复杂度是多少，假设使用的是树形通信结构。

（3） 假设串行程序的运行时间为$T_{serial}=k*n$，并行程序的运行时间为：$T_{parallel}(n,p)=\frac{k*n}{p}+c*log_2(p)$,其中$c$为常数，请推导出加速比$S(n,p)$和效率$E(n,p)$的表达式，当$n$固定，$p$增大时，效率会如何变化，为什么。

**答案：**

（1） $n$能被$p$整除，假设$n=p*m$,那么每个进程负责$m=\frac{n}{p}$个梯形，起始点为$x_{start}=a+i*m*h$,结束点为$x_{end}=a+(i*m+m-1)*h$,若不能整除，假设$n=p*m+r$，那前$r$个进程负责$m+1$个梯形，其余$p-r$个进程负责$m$个梯形。

（2） 使用 MPI_Reduce 使用树形通信结构时，在每一步，参与通信的进程对数减半，总共需要$log_2 p$步才能将数据汇聚到跟进程，时间复杂度为$O(log_2 p)$。

（3） 加速比$S(n,p)=\frac{T_{serial}}{T_{parallel}}=\frac{k*n}{\frac{k*n}{p}+c*log_2(p)}=\frac{k*n*p}{k*n+c*p*log_2(p)}$,效率比$E(n,p)=\frac{S(n,p)}{p}=\frac{k*n}{k*n+c*p*log_2(p)}$，当$n$固定，$p$增大时，效率会显著下降，观察效率比的计算公式，当$p$增大时，分母变大，导致计算效率降低，这是因为虽

然增加进程数减少了单进程的计算时间，但通信开销在总时间中的占比越来越大，最终导致效率降低。

# 作业三

一、 考虑以下两个计算 1 到 N 之间所有整数平方和的 Pthreads 程序片段。假设 N = $10^9$，线程数 thread_count = 8

**代码段 A：**

```
void* Square_Sum_A(void* rank) {
    long my_rank = (long) rank;
    long long local_n = N / thread_count;
    long long first = my_rank * local_n + 1;
    long long last = first + local_n;
    for (long long i = first; i < last; i++) {
        pthread_mutex_lock(&mutex);
        global_sum += i * i;
        pthread_mutex_unlock(&mutex);
    }
    return NULL;
}
```

**代码段 B：**

```
void* Square_Sum_B(void* rank) {
    long my_rank = (long) rank;
    long long local_n = N / thread_count;
    long long my_sum = 0;
    long long first = my_rank * local_n + 1;
    long long last = first + local_n;
    for (long long i = first; i < last; i++) {
        my_sum += i * i;
    }
    pthread_mutex_lock(&mutex);
```

```
    global_sum += my_sum;
    pthread_mutex_unlock(&mutex);
    return NULL;
}
```

回答下列问题：

（1）指出代码段 A 中的临界区，并估算在 $N=10^9$ 时，代码段 A 总共执行了多少次锁操作

（2）从操作系统调度的角度分析，为什么代码段 A 会导致 CPU 利用率虚高但实际计算进度缓慢

（3）在代码段 B 中，主线程在创建线程后立即调用 pthread_join（阻塞调用它的线程，直到指定的线程结束），请说明 pthread_join 在此处的作用

参考答案：

（1）临界区是代码行 global_sum += i * i；这条语句位于循环体内，且循环执行了 N 次，因此总共会发生 N 次（即 $10^9$ 次）pthread_mutex_lock 和相同次数的 pthread_mutex_unlock

（2）代码段 A 对临界区的执行次数过多，当一个线程持有锁时，其余线程会被迫挂起进入休眠，或在等待队列中空转。一旦当前线程释放锁，操作系统调度器必须从休眠队列中唤醒另一个线程，这涉及昂贵的上下文切换操作：保存当前线程的寄存器、切换内核栈、更新页表基址等 。CPU 绝大部分周期都消耗在维护一致性和处理调度事务上，而非有效计算

（3）pthread_join 充当了路障。它阻塞调用线程（通常是主线程），直到目标线程执行完毕并返回，主线程可以确保在打印 global_sum 最终结果之前，所有线程均已完成了最后一步 global_sum += my_sum 的更新

二、读写锁允许多个读者同时进入临界区，但写者必须互斥访问 。考虑以下一个简单的、基于信号量实现的自定义读写锁逻辑：

```
int read_count = 0;
sem_t count_mutex; // 保护 read_count 的更新，初始化为 1
sem_t rw_mutex;      // 保护共享资源的写操作，初始化为 1
```

```
void* reader(void* arg) {
    sem_wait(&count_mutex);
    read_count++;
    if (read_count == 1)
        sem_wait(&rw_mutex); //  第一个读者锁定资源
    sem_post(&count_mutex);

    //执行读操作

    sem_wait(&count_mutex);
    read_count--;
    if (read_count == 0)
        sem_post(&rw_mutex); //  最后一个读者释放资源
    sem_post(&count_mutex);
    return NULL;
}

void* writer(void* arg) {
    sem_wait(&rw_mutex);

    //  执行写操作

    sem_post(&rw_mutex);
    return NULL;
}
```

回答下列问题：

（1） 描述在该实现下，什么情况下写者可能会遭遇“饥饿”现象？
（2） 该实现倾向于“读者”还是“写者”？如何避免对写者的饥饿问题，简要描述你的方案

参考答案：

（1） 该实现中只要有读者在临界区内，后续到来的读者就可以直接进入，写者则会被无限期挂起。每当一个读者离开前都有一个新的读者到达，导致 read_count 永远不会减至 0，rw_mutex 永远不会被释放，写者永久处于阻塞状态，产生饥饿。
（2） 该实现倾向于读者。为了解决写者饥饿，可以引入一个额外的信号量，当写者到达时，锁定该信号量，阻止后续新读者进入。一旦当前临界区内的旧读者全部离开，写者就能立即获得 rw_mutex，而新读者必须在写者完成之后才能进入

三、某多线程程序中有两个共享数据结构 A 和 B，分别由互斥锁 mutexA 和 mutexB 保护。程序启动后有两个线程 T0 和 T1 并发执行，它们的加锁顺序如下：

线程 T0：先执行 pthread_mutex_lock(&mutexA)，再执行 pthread_mutex_lock(&mutexB)

线程 T1：先执行 pthread_mutex_lock(&mutexB)，再执行 pthread_mutex_lock(&mutexA)

假设系统中出现如下执行时序：

时刻 0：T0 成功获得 mutexA，T1 成功获得 mutexB

时刻 1：T0 试图获得 mutexB，T1 试图获得 mutexA

请回答下列问题：

（1）此时程序会发生什么现象？请说明原因。

（2）为什么这种问题在线程程序中比较危险？

（3）给出一种简单可行的修改方法，避免该问题再次发生。

参考答案：

（1）此时程序会发生**死锁**。

原因是：

T0 已经持有 mutexA，正在等待 mutexB

T1 已经持有 mutexB，正在等待 mutexA

于是形成了“互相等待”的循环：

T0 不释放 mutexA，因为它还没拿到 mutexB

T1 不释放 mutexB，因为它还没拿到 mutexA

两个线程都会一直阻塞下去，程序无法继续向前执行。

（2）程序不会正常结束，也不会继续完成任务；线程会一直卡住，占用系统资源；如果程序比较大，死锁位置可能很隐蔽，调试困难；在服务器程序或长期运行程序中，死锁可能直接导致系统无响应。

（3）一种简单可行的方法是：

规定所有线程必须按照相同顺序获取互斥锁。

例如统一规定：

不管哪个线程，凡是同时需要 mutexA 和 mutexB，都必须先锁 mutexA，再锁 mutexB

这样就不会出现：

一个线程先拿 A 再等 B

另一个线程先拿 B 再等 A

也就不会形成循环等待，从而避免死锁

四、阅读以下代码片段，回答问题

```
#include <pthread.h>
#include <stdio.h>

int counter = 0; // 全局计数器

void* UnsafeIncrement(void* arg) {
    for (int i = 0; i < 100000; i++) {
        counter++;
    }
```

```
    return NULL;
}

int main() {
    pthread_t t1, t2;

    pthread_create(&t1, NULL, UnsafeIncrement, NULL);
    pthread_create(&t2, NULL, UnsafeIncrement, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Final Counter Value: %d\n", counter);
    return 0;
}
```

（1）理论上，如果程序正确运行，counter 的最终值应该是多少？为什么实际运行结果通常小于这个理论值？
（2）请指出 counter++ 操作在汇编层面通常包含哪三个步骤？并解释这如何导致数据竞争。
（3）如何修改 UnsafeIncrement 函数，使其线程安全

参考答案：
（1）程序正确运行的话，counter 的最终值应该为 20000，但是实际结果通常小于 20000，因为 counter++不是原子操作，当两个线程同时读取更新的时候，其中一次更新会被覆盖，导致计数减一。
（2）通常会分解为三个步骤，分别是读取，更新，写回，导致数据竞争的情况为，两个线程同时读取，计数完之后同时写回，导致更新丢失。
（3）使用锁使计数成为原子操作

```
void* SafeIncrement(void* arg) {
    for (int i = 0; i < 100000; i++) {
        pthread_mutex_lock(&mutex); //  加锁
        counter++;                  //  访问临界区
        pthread_mutex_unlock(&mutex); //  解锁
    }
    return NULL;
}
```

五、假设我们有一个多线程程序，用于统计两类事件发生的次数。两个线程分别只更新自己对应的计数器

```
#include <pthread.h>
#include <stdio.h>
```

```
// 定义一个结构体，包含两个计数器
typedef struct {
    long counter_a; // 线程 0 专门累加这个
    long counter_b; // 线程 1 专门累加这个
} CounterPair;

CounterPair counters;

void* ThreadFunc(void* arg) {
    long tid = (long)arg;
    // 模拟大量工作
    for (int i = 0; i < 100000000; i++) {
        if (tid == 0) {
            counters.counter_a++; // 线程 0 操作
        } else {
            counters.counter_b++; // 线程 1 操作
        }
    }
    return NULL;
}

int main() {
    pthread_t t0, t1;
    // 创建两个线程
    pthread_create(&t0, NULL, ThreadFunc, (void*)0);
    pthread_create(&t1, NULL, ThreadFunc, (void*)1);

    pthread_join(t0, NULL);
    pthread_join(t1, NULL);

    printf("A: %ld, B: %ld\n", counters.counter_a, counters.counter_b);
    return 0;
}
```

（1） 为什么上述代码可能存在严重的伪共享问题，并解释一下为何在两个线程上操作不同的变量，却会导致性能急剧下降？

（2） 为了消除伪共享，请修改 CounterPair 结构体的定义，利用填充技术确保 counter_a 和 counter_b 位于不同的缓存行中。假设系统的缓存行大小为 64 字节。

（3） 除了填充结构体外，还有另一种利用局部性原理的高性能方案，如何修改 ThreadFunc 函数的逻辑，让每个线程先在“私有”的内存空间进行累加，最后再合并结果，从而彻底避开伪共享？

参考答案：

（1） 在给定的代码中，counter_a 和 counter_b 是连续定义在结构体中的。在内存中，它们很可能是相邻存储的，如果这两个变量的地址位于同一个 CPU 缓存行内，就会发生伪共享。假设线程 0 和线程 1 所在的核心都读取了包含这两个变量的缓存行，状态为共享当线程 0 尝试修改 counter_a 时，它必须将缓存行状态改为修改，这会触发监听协议，迫使线程 1 所在核心中的该缓存行副本失效，随后，当线程 1 需要修改 counter_b 时，它发现缓存行已失效，必须重新从内存或其他核心读取数据。这一过程反复进行，导致两个线程虽然操作的是不同变量，却像在串行执行一样频繁交换缓存行，极大地浪费了总线带宽和 CPU 时间。

（2）typedef struct {
    long counter_a;
    char padding[64]; // 填充一个完整的缓存行大小
    long counter_b;
} CounterPair;

（3）每个线程不直接更新全局结构体，而是先在栈上或线程私有的内存中进行累加。只有在最后，才将私有的累加结果写回一次到共享内存中。

```
void* ThreadFunc(void* arg) {
    long tid = (long)arg;
    long local_sum = 0; // 私有局部变量，位于线程栈上，无共享

    // 1. 线程先在私有变量中进行所有计算
    for (int i = 0; i < 100000000; i++) {
        local_sum++; // 只操作局部变量，无伪共享
    }

    // 2. 最后，将结果写入共享变量（仅一次操作）
    if (tid == 0) {
        counters.counter_a = local_sum;
    } else {
        counters.counter_b = local_sum;
    }
    return NULL;
}
```

# 作业四

一、 并行化时我们需要注意并识别循环依赖，分析以下两个 C 语言循环片段，判断其是否可以直接使用 #pragma omp parallel for 进行并行化。如果不能，请说明原因：

**片段 A：**

```
for (i = 1; i < n; i++) {
    A[i] = A[i-1] + B[i];
}
```

**片段 B：**

```
int index = 0;
for (i = 0; i < n; i++) {
    index = (i + offset) % n; //offset 为一个固定的常数
    C[index] = A[i] * B[i];
}
```

参考答案：

片段 A 不能直接并行化，其第 i 次的迭代结果依赖于第 i-1 次迭代的结果，如果并行化，例如，线程 1 负责 i=1...500，线程 2 负责 i=501...1000。当线程 2 开始执行 i=501 时，它需要 A[500] 的值，但此时线程 1 可能尚未计算出该值，导致计算结果不可预计

片段 B 并行化时需要保证 index 有私有作用域，否则如果 index 被声明为共享变量，那么所有线程都会竞争写入同一个 index 内存单元，导致出现错误，因此需要添加一个 private 子句 private(index)到 parallel 指令中

二、请阅读以下使用 OpenMP 并行化的 C 语言代码片段，假设 $n$ 非常大，线程数为 4 个：

```
double result = 0.0;
// 使用 OpenMP 并行化
#pragma omp parallel for reduction(+: result)
for (int i = 0; i < n; i++) {
    result += HeavyCompute(i);
}
```

其中 HeavyCompute(i) 是一个模拟函数，$i$ 越大，计算所需的时间越长

（1） 虽然这段代码计算结果是正确的，但在性能上可能会出现什么问题？

（2） OpenMP 的 parallel for 默认使用哪种调度策略？为什么这种策略在上述场景下效率低下？

（3） 请修改 #pragma 指令，使用一种更合适的调度策略来平衡负载，提高程序运行速度。

参考答案：

（1） 会出现负载不均衡现象，由于循环是顺序分配的，前面的负责小的 $i$线程会很快完成它们的任务，然后进入空闲等待状态。后面负责大的 $i$的线程会因为计算量大，工作很长时间。结果导致整个程序的运行时间取决于最慢的那个线程，导致 CPU 资源浪费。

（2） OpenMP 的 parallel for 默认通常使用 static 调度，static 调度会在循环开始前，将迭代次数平均切分给各个线程，它假设每次迭代的计算量是相同的，但在本题中，计算量随 $i$ 增大而增加，导致分配给后半部分迭代的线程负载过重，负载不均导致效率低下

（3） 应该使用 dynamic 调度 或 guided 调度

```
//动态调度，块大小设为 100（或一个较小的数）
#pragma omp parallel for reduction(+: result) schedule(dynamic, 100)
for (int i = 0; i < n; i++) {
    result += HeavyCompute(i);
}
```

虽然会有少量的调度开销，但它能确保所有线程都在忙碌，直到所有繁重的计算任务被瓜分完毕，从而显著缩短总运行时间。

三、请阅读以下代码片段并回答问题

```
#include <stdio.h>
#include <omp.h>
int main() {
```

```
    int i, s = 0, a = 5;
    #pragma omp parallel num_threads(3) private(i) shared(a)
reduction(+:s)
    {
        int tid = omp_get_thread_num();
        a += tid;
        #pragma omp for
        for (i = 1; i <= 6; i++) {
            s += i;
        }
    }
    printf("s = %d\n", s);
    printf("a = %d\n", a);
    return 0;
}
```

（1） 变量 i, s, a 分别是什么变量？

（2） 指出这段代码所存在的问题。

（3） 分析最终 a 和 s 的输出。

参考答案：

（1） i 是私有变量每个线程独立，s 是规约变量，每个线程私有，最后合并，a 是共享变量，所有线程共享同一份变量。

（2） 因为 a 是共享变量，多个线程同时读写 a，会出现数据竞争情况，最终可能导致 a 的结果不确定

（3） S 的值无数据竞争问题，按照循环计算完成后即为 1+2+3+4+5+6=21，a 的值若无竞争情况出现，则应该为 5+0+1+2=8，如果出现了数据竞争的情况，会出现读写丢失的情况，那么最后的情况可能为 6 或 7 或 8。

四、某程序使用 OpenMP 并行化如下循环：

```
#pragma omp parallel for num_threads(4)
for (int i = 0; i < 16; i++) {
    work(i);
}
```

设系统采用默认调度方式，并假设每次迭代 work(i) 的执行时间基本相同。

请回答下列问题：

（1）若 16 次迭代被连续地、尽可能均匀地分配给 4 个线程，请写出每个线程可能负责的迭代编号范围。

（2）这种分配方式有什么优点？

（3）如果不同迭代的计算量差异非常大，那么默认这种分配方式可能会带来什么问题？

参考答案：

（1）若按连续且均匀的方式分配，16 次迭代由 4 个线程处理，则每个线程负责 4 次迭代。一种可能的分配为：

线程 0：i = 0, 1, 2, 3
线程 1：i = 4, 5, 6, 7
线程 2：i = 8, 9, 10, 11
线程 3：i = 12, 13, 14, 15

（2）这种分配方式的优点包括：

分配规则简单，开销小；

每个线程处理连续区间，便于实现；

当各次迭代耗时接近时，能够较好实现负载均衡。

（3）如果不同迭代的计算量差异很大，则可能出现**负载不均衡**问题。
例如某个线程分到的迭代特别“重”，而另一些线程分到的迭代较“轻”，就会导致：

有些线程先完成后处于空闲；

其他线程仍在工作；

最终整体并行效率下降。

五、阅读下面的 OpenMP 程序片段：

```
double x = 0.0;
double y = 0.0;

#pragma omp parallel num_threads(4)
{
    for (int i = 0; i < 1000; i++) {
        #pragma omp atomic
        x += 1.0;

        #pragma omp atomic
        y += 1.0;
    }
}
```

请回答下列问题：

（1）这两条 atomic 指令分别保护的是什么操作？

（2）对变量 x 的更新和对变量 y 的更新，是否会被视为同一个“临界区”？为什么？

（3）如果把两条 atomic 改成一个 critical 代码块，将两次加法都放进去，程序的并发性会发生什么变化？

参考答案：

（1）第一条 atomic 保护的是：x += 1.0;

第二条 atomic 保护的是：y += 1.0;

每条 atomic 只保证对应那一条更新语句是原子的，不会被多个线程同时破坏。

（2）不会被视为同一个“临界区”。

原因是：

atomic 的保护范围通常只针对紧跟在后面的那一条内存更新语句；

第一条 atomic 只保护对 x 的更新；

第二条 atomic 只保护对 y 的更新；

因此它们不是一个整体的临界区，而是两个彼此独立的原子更新操作。

（3）如果改成一个 critical 代码块，例如：

```
#pragma omp critical
{
    x += 1.0;
    y += 1.0;
}
```

那么线程在进入该代码块后，需要一次性独占执行其中两条语句。这样会导致：

并发性下降；

线程之间串行化程度更高；

一般来说开销会比两条独立的 atomic 更大。

但好处是：如果程序逻辑要求“更新 x 和更新 y 必须作为一个整体执行”，那么 critical 更合适。

# 作业五

一、 CUDA 程序中常见的内存类型包括 全局内存、共享内存、寄存器、本地内存、常量内存等。请结合下面代码，回答以下问题：

```
__constant__ float c_alpha;

__device__ float d_total;

__global__ void memoryDemo(float *g_in, float *g_out, int n) {
    __shared__ float s_tile[256];

    int tid = threadIdx.x;
    int idx = blockIdx.x * blockDim.x + threadIdx.x;

    float temp = 0.0f;
    float localArray[4];

    if (idx < n) {
        s_tile[tid] = g_in[idx];

        __syncthreads();

        temp = s_tile[tid] * c_alpha;
        localArray[0] = temp;

        g_out[idx] = localArray[0];

        atomicAdd(&d_total, temp);
    }
}
```

1. s_tile 属于哪类内存类型，简要描述该类内存的访问范围与生命周期
2. c_alpha 属于哪类内存？适合存放什么数据？

参考答案：

1. s_tile 使用了 __shared__ 修饰，属于共享内存，共享内存是一个 block 内部线程共享的片上存储空间，同一个 block 中的线程可以通过它交换数据，不同 block 之间不能直接共享。它的生命周期只在当前 block 执行期间有效，block 执行结束后对应的共享内存内容也随之失效
2. c_alpha 使用了 __constant__ 修饰，属于常量内存，适合存储小的只读数据，以及许多线程都会读取的常量

二、 阅读下面 CUDA 程序片段，回答以下问题：

```
__global__ void vecAdd(float *a, float *b, float *c, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    c[i] = a[i] + b[i];
}

int main() {
    int n = 1000;
    size_t size = n * sizeof(float);

    float *d_a, *d_b, *d_c;

    cudaMalloc((void**)&d_a, size);
    cudaMalloc((void**)&d_b, size);
    cudaMalloc((void**)&d_c, size);

    int threads = 256;
    int blocks = (n + threads - 1) / threads;

    vecAdd<<<blocks, threads>>>(d_a, d_b, d_c, n);
  }
```

1. 这段核函数存在什么潜在的错误？
2. 当 n = 1000 时，一共启动多少个线程？是否可能发生对数组的越界访问？

参考答案：

1. 核函数内 c[i] = a[i] + b[i]没有进行边界判断，当线程总数大于数组长度时，一些线程会访问数组范围之外的地址
2. 计算 block 数量：(1000 + 256 - 1) / 256 = 4，因此启动的线程总数为 4 * 256 = 1024。由于数组长度为 1000，当 i = 1000, 1001, ..., 1023 时，线程仍然会执行，将会访问越界地址

三、 假设有一个 $N \times N$ 的矩阵 M 存储在一维数组 float* mat 中（按行优先存储），要在 GPU 上做矩阵转置，kernel 如下：

```
__global__ void Transpose(float* mat, float* matT, int N) {
        int row = blockIdx.y * blockDim.y + threadIdx.y;
        int col = blockIdx.x * blockDim.x + threadIdx.x;

        if (row < N && col < N) {
            matT[col * N + row] = mat[row * N + col];
        }
}
```

主机端启动方式：

```
dim3 blockSize(16, 16);
```

```
dim3 gridSize( (N + blockSize.x - 1)/blockSize.x,
               (N + blockSize.y - 1)/blockSize.y );

Transpose<<<gridSize, blockSize>>>(mat, matT, N);
```

请回答：

1. 如果 N = 1000，计算 gridSize.x 和 gridSize.y 各是多少？
2. 假设 GPU 有 1024 个线程每个 SM 最多可以同时执行，blockSize 设为 (16,16) 后，每个 block 占用多少个线程？最后一个 block 中可能有多少线程 不执行计算？
3. 如果去掉 kernel 中的边界判断 if(row<N && col<N)，会发生什么问题？为什么这个判断很重要？

参考答案：

1. gridSize.x=⌈1000/16⌉=63　gridSize.y=⌈1000/16⌉=63

2. 每个 block 线程数 = 16 × 16 = 256

最后一个 block 可能处理的有效行/列数为：

有效行数 = 1000 − 16×62 = 8

有效列数 = 1000 − 16×62 = 8

有效线程数 = 8 × 8 = 64

不执行计算线程 = 256 − 64 = 192

3. 去掉边界判断的后果：

1) 会访问数组越界，写入 matT 的非法索引
2) 可能导致结果错误或 kernel 崩溃
3) 判断保证线程只处理合法的矩阵元素

四、 假设有一个 N×N 的矩阵 M 存储在一维数组 float* mat 中，按行优先方式存储。现在希望在 GPU 上实现“把矩阵中每个元素都乘以 2”的操作，kernel 框架如下：

```
__global__ void MatScale2(float* mat, int N) {
int row = blockIdx.y * blockDim.y + threadIdx.y;
int col = blockIdx.x * blockDim.x + threadIdx.x;

if (_____) {
    mat[_____] = 2.0f * mat[_____];
}

}
```

主机端启动方式如下：

```
dim3 blockSize(16, 16);
dim3 gridSize((N + blockSize.x - 1) / blockSize.x,
(N + blockSize.y - 1) / blockSize.y);

MatScale2<<<gridSize, blockSize>>>(mat, N);
```

请完成下列问题：

1. 如果某线程满足 row=3，col=5，那么它处理的是矩阵中的哪一个元素？
2. 补全上面的 kernel，使每个线程对自己负责的矩阵元素乘以 2
3. 若 N=1000，计算 gridSize.x 和 gridSize.y 的值，并说明为什么 kernel 中必须写边界判断

参考答案：

1. 线程处理的是矩阵第 3 行第 5 列的元素。
   由于矩阵按行优先方式存储，它在一维数组中的下标为：
   3 * N + 5

2. 补全后的 kernel：

```
__global__ void MatScale2(float* mat, int N) {
int row = blockIdx.y * blockDim.y + threadIdx.y;
int col = blockIdx.x * blockDim.x + threadIdx.x;

if (row < N && col < N) {
      mat[row * N + col] = 2.0f * mat[row * N + col];
           }

}
```

3. gridSize.x = (1000 + 16 - 1) / 16 = 63
   gridSize.y = (1000 + 16 - 1) / 16 = 63

必须写边界判断的原因是：
当 N 不能刚好被 blockSize 整除时，最后一行或最后一列的某些线程计算出的 row 或 col 可能会超出矩阵范围。
如果不判断：会访问非法下标、可能导致数组越界、可能产生错误结果，严重时会导致 kernel 执行失败